

Principles of Computer Science II

Dynamic Programming and Sequence Similarity

Marco Zecchini

Sapienza University of Rome

Lecture 9

Coin Change Problem

United States Change Problem:

Convert some amount of money into the fewest number of coins.

Input: An amount of money, M , in cents.

Output: The smallest number of quarters q , dimes d , nickels n , and pennies p whose values add to M (i.e., $25q + 10d + 5n + p = M$ and $q + d + n + p$ is as small as possible).

- We know that the greedy solution is not good..
- ... we are only left with a brute-force approach that is impractical

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;
- the best combination for $77 - 3 = 74$ cents, plus a 3-cent coin;

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;
- the best combination for $77 - 3 = 74$ cents, plus a 3-cent coin;
- the best combination for $77 - 7 = 70$ cents, plus a 7-cent coin.

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;
- the best combination for $77 - 3 = 74$ cents, plus a 3-cent coin;
- the best combination for $77 - 7 = 70$ cents, plus a 7-cent coin.

$$bestNumCoins_M = \min \begin{cases} bestNumCoins_{M-1} + 1 \\ bestNumCoins_{M-3} + 1 \\ bestNumCoins_{M-7} + 1 \end{cases}$$

Same for 76 and, then, 75...

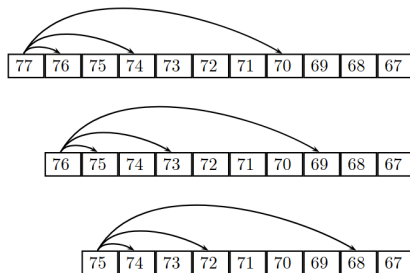


Figure 6.1 The relationships between optimal solutions in the Change problem. The smallest number of coins for 77 cents depends on the smallest number of coins for 76, 74, and 70 cents; the smallest number of coins for 76 cents depends on the smallest number of coins for 75, 73, and 69 cents, and so on.

RecursiveChange

```
RECURSIVECHANGE( $M, \mathbf{c}, d$ )
1  if  $M = 0$ 
2      return 0
3   $bestNumCoins \leftarrow \infty$ 
4  for  $i \leftarrow 1$  to  $d$ 
5      if  $M \geq c_i$ 
6           $numCoins \leftarrow \text{RECURSIVECHANGE}(M - c_i, \mathbf{c}, d)$ 
7          if  $numCoins + 1 < bestNumCoins$ 
8               $bestNumCoins \leftarrow numCoins + 1$ 
9  return  $bestNumCoins$ 
```

RecursiveChange

```
RECURSIVECHANGE( $M, \mathbf{c}, d$ )
1  if  $M = 0$ 
2      return 0
3   $bestNumCoins \leftarrow \infty$ 
4  for  $i \leftarrow 1$  to  $d$ 
5      if  $M \geq c_i$ 
6           $numCoins \leftarrow \text{RECURSIVECHANGE}(M - c_i, \mathbf{c}, d)$ 
7          if  $numCoins + 1 < bestNumCoins$ 
8               $bestNumCoins \leftarrow numCoins + 1$ 
9  return  $bestNumCoins$ 
```

- However, it computes the optimum amount of coin repeatedly

RecursiveChange

```
RECURSIVECHANGE( $M, \mathbf{c}, d$ )
1  if  $M = 0$ 
2      return 0
3   $bestNumCoins \leftarrow \infty$ 
4  for  $i \leftarrow 1$  to  $d$ 
5      if  $M \geq c_i$ 
6           $numCoins \leftarrow \text{RECURSIVECHANGE}(M - c_i, \mathbf{c}, d)$ 
7          if  $numCoins + 1 < bestNumCoins$ 
8               $bestNumCoins \leftarrow numCoins + 1$ 
9  return  $bestNumCoins$ 
```

- However, it computes the optimum amount of coin repeatedly
- The optimal coin combination for 70 cents is recomputed repeatedly nine times over and over as
(77 - 7), (77 - 3 - 3 - 1), (77 - 3 - 1 - 3), (77 - 1 - 3 - 3), (77 - 3 - 1 - 1 - 1 - 1), (77 - 1 - 3 - 1 - 1 - 1), (77 - 1 - 1 - 3 - 1 - 1), (77 - 1 - 1 - 1 - 3 - 1), (77 - 1 - 1 - 1 - 1 - 3) and (77 - 1 - 1 - 1 - 1 - 1 - 1 - 1).

Reverse the order...

Leverage previously computed solutions to form solutions to larger problems and avoid all this recomputation

```
DPCHANGE( $M, c, d$ )  
1   $bestNumCoins_0 \leftarrow 0$   
2  for  $m \leftarrow 1$  to  $M$   
3       $bestNumCoins_m \leftarrow \infty$   
4      for  $i \leftarrow 1$  to  $d$   
5          if  $m \geq c_i$   
6              if  $bestNumCoins_{m-c_i} + 1 < bestNumCoins_m$   
7                   $bestNumCoins_m \leftarrow bestNumCoins_{m-c_i} + 1$   
8  return  $bestNumCoins_M$ 
```

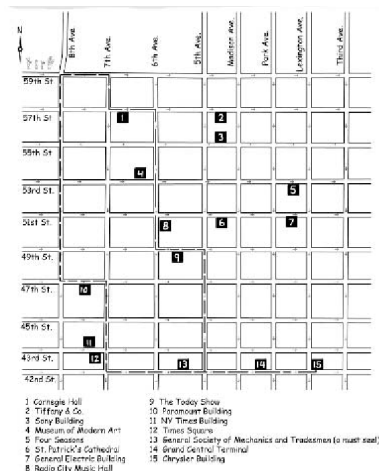
We compute a solution for each possible amount of money m from 1 to M and, since it takes d steps, to find the right coin the cost is $\mathcal{O}(Md)$.

Dynamic Programming

- Dynamic programming solves problems by combining the solutions to subproblems.
- “Programming” refers to a tabular method, not to writing computer code.
- Dynamic programming applies when the subproblems overlaps - that is, when subproblems share subsubproblems
- A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem

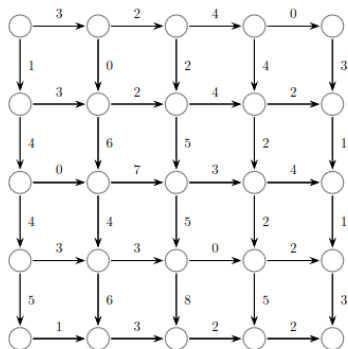
Manhattan Tourist Problem

- Sightseeing tour in Manhattan where a group of tourists wants to walk from the corner of 59th Street and 8th Avenue to the Chrysler Building.
- Many attractions along the way and the tourists want to see as many attractions as possible.
- The tourists can move either to the south or to the east, but even so, they can choose from many different paths.



Manhattan Tourist Problem as a graph

- We can represent this gridlike structure as a *graph*
- Intersections are *vertexes*
- Streets are *edges* that are oriented either towards south ($\downarrow$) or east ($\rightarrow$) and have a *weight*
- A *path* is a continuous sequence of edges, and the *length of a path* is the sum of the edge weights in the path



Problem statement

Manhattan Tourist Problem:

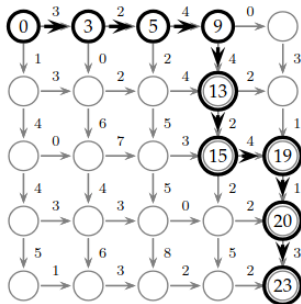
Find a longest path in a weighted grid.

Input: A weighted grid G with two distinguished vertices:
a source and a sink.

Output: A longest path in G from source to sink.

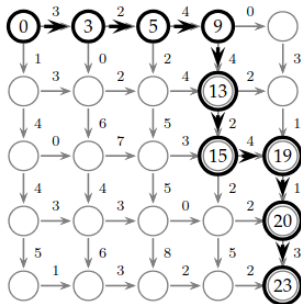
Greedy approach

- Always pick the maximum edge for each vertex



Greedy approach

- Always pick the maximum edge for each vertex
- Can we do better?

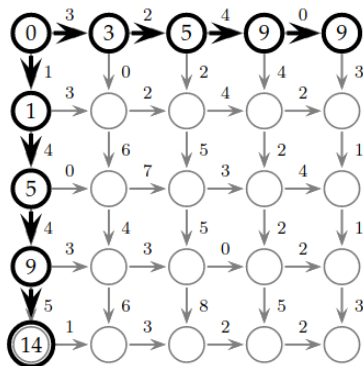


DP Approach: easier subproblems

- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.

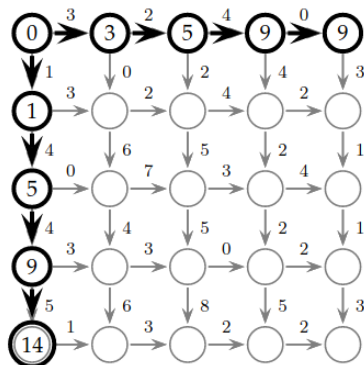
DP Approach: easier subproblems

- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.
- Let $s_{i,j}$ denotes the optimal solution for the vertex (i, j) .



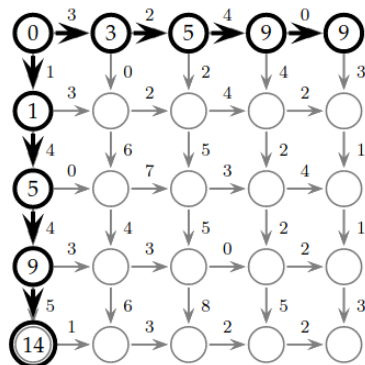
DP Approach: easier subproblems

- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.
- Let $s_{i,j}$ denotes the optimal solution for the vertex (i, j) .
- Finding $s_{0,j}$ (for $0 \leq j \leq m$) is not hard because tourists cannot choose an arbitrary path: they can only go east.

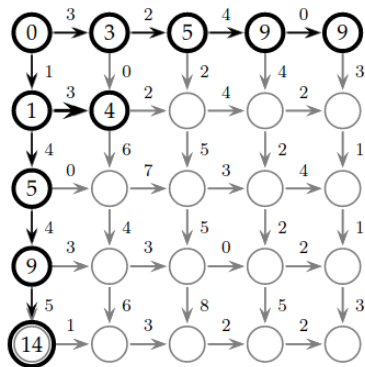


DP Approach: easier subproblems

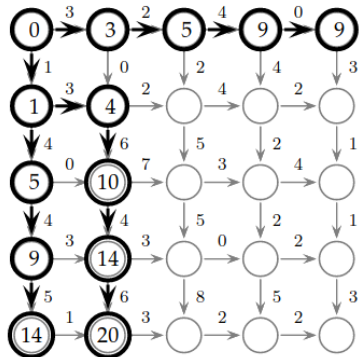
- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.
- Let $s_{i,j}$ denotes the optimal solution for the vertex (i, j) .
- Finding $s_{0,j}$ (for $0 \leq j \leq m$) is not hard because tourists cannot choose an arbitrary path: they can only go east.
- The same for $s_{i,0}$ (for $0 \leq i \leq n$).



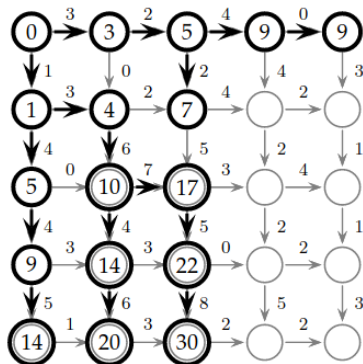
- Let us find the solution for $s_{1,1}$.
- Tourist can arrive to $(1, 1)$ in only two ways: from $(0, 1)$ or from $(1, 0)$
- The best path will be: either (1) $s_{0,1} +$ the weight from $(0, 1)$ to $(1, 1)$ or (2) $s_{1,0} +$ the weight from $(1, 0)$ to $(1, 1)$.
- We take the largest value.



- Similar logic to $s_{2,1}$, $s_{3,1}$ and so on.

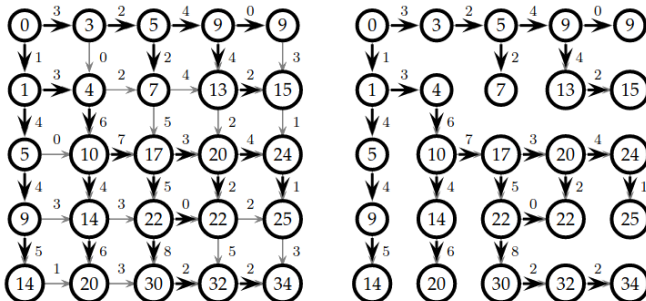


- Let us find the solution for $s_{1,2}$.
- Tourist can arrive to $(1, 2)$ in only two ways: from $(1, 1)$ or from $(2, 0)$
- The best path will be: either (1) $s_{1,1}$ + the weight from $(1, 1)$ to $(1, 2)$ or (2) $s_{0,2}$ + the weight from $(0, 2)$ to $(1, 2)$.
- We take the largest value.
- Similar logic to $s_{2,2}$, $s_{3,2}$ and so on.



We can compute the optimal solution for each vertex with this approach.

$$s_{i,j} = \max \left\{ \begin{array}{l} s_{i-1,j} + \text{weight of the edge between } (i-1,j) \text{ and } (i,j) \\ s_{i,j-1} + \text{weight of the edge between } (i,j-1) \text{ and } (i,j) \end{array} \right.$$



DP Approach: the algorithm

- $\downarrow$ **w** two-dimensional array representing the weights of the grid's edges that run north to south
- $\rightarrow$ **w** is a two-dimensional array representing the weights of the grid's edges that run west to east.

```

MANHATTANTOURIST( $\overleftarrow{w}, \overrightarrow{w}, n, m$ )
1   $s_{0,0} \leftarrow 0$ 
2  for  $i \leftarrow 1$  to  $n$ 
3       $s_{i,0} \leftarrow s_{i-1,0} + \overleftarrow{w}_{i,0}$ 
4  for  $j \leftarrow 1$  to  $m$ 
5       $s_{0,j} \leftarrow s_{0,j-1} + \overrightarrow{w}_{0,j}$ 
6  for  $i \leftarrow 1$  to  $n$ 
7      for  $j \leftarrow 1$  to  $m$ 
8           $s_{i,j} \leftarrow \max \begin{cases} s_{i-1,j} + \overleftarrow{w}_{i,j} \\ s_{i,j-1} + \overrightarrow{w}_{i,j} \end{cases}$ 
9  return  $s_{n,m}$ 

```

Initial Conditions

Recurrence

DP Approach: the algorithm

- $\downarrow$
 $\mathbf{w}$ two-dimensional array representing the weights of the grid's edges that run north to south
- $\rightarrow$
 $\mathbf{w}$ is a two-dimensional array representing the weights of the grid's edges that run west to east.
- The cost is $\mathcal{O}(nm)$

```

MANHATTANTOURIST( $\overleftarrow{\mathbf{w}}, \overrightarrow{\mathbf{w}}, n, m$ )
1   $s_{0,0} \leftarrow 0$ 
2  for  $i \leftarrow 1$  to  $n$ 
3       $s_{i,0} \leftarrow s_{i-1,0} + \overleftarrow{w}_{i,0}$ 
4  for  $j \leftarrow 1$  to  $m$ 
5       $s_{0,j} \leftarrow s_{0,j-1} + \overrightarrow{w}_{0,j}$ 
6  for  $i \leftarrow 1$  to  $n$ 
7      for  $j \leftarrow 1$  to  $m$ 
8           $s_{i,j} \leftarrow \max \begin{cases} s_{i-1,j} + \overleftarrow{w}_{i,j} \\ s_{i,j-1} + \overrightarrow{w}_{i,j} \end{cases}$ 
9  return  $s_{n,m}$ 

```

Initial Conditions

Recurrence

Steps to design a DP algorithm.

Dynamic programming typically applies to *optimization problems*: each solution has a value (i.e., it is a number) and you want to find a solution with the optimal (minimum or maximum) value. To develop a dynamic-programming algorithm, follow a sequence of four steps:

- ① Characterize the structure of an optimal solution.
- ② Recursively define the value of an optimal solution.
- ③ Compute the value of an optimal solution, typically in a bottom-up fashion.
- ④ Construct an optimal solution from computed information.